

Python Essentials III — Control Flow, Functions & Modules

By the end of today your programs will make decisions, repeat actions automatically, and be organised into reusable building blocks called functions. This is the day your code starts feeling like a real program, not just a list of instructions.

Setup: Open VS Code. Create a new file called `day4_practice.py`. As always — type every example. Do not copy-paste.

SECTION 01 Control Flow — Making Your Program Take Decisions

So far, every program you have written runs from top to bottom, one line at a time, no matter what. But real programs are smarter than that. A banking app should behave differently when you have enough balance versus when you don't. A login page should react differently to a correct password versus a wrong one.

Control flow is the ability of a program to make decisions and choose which lines of code to run based on certain conditions. It is what separates a real program from a simple script.

SECTION 02 if, elif, else — Teaching Python to Make Decisions

if / elif / else

What it is: A structure that checks conditions one by one and runs a block of code only when a condition is True. `if` checks the first condition. `elif` (else if) checks the next condition if the first was False. `else` runs when all conditions above were False.

Real-life analogy: Think of a traffic light. IF the light is green, go. ELSE IF the light is orange, slow down. ELSE (it must be red), stop. Python checks each condition in order and acts accordingly.

In Python: `if age >= 18: (check condition, run this block if True)`

The basic if statement

The simplest decision: check one condition. If it is True, run the indented block. If it is False, skip it and move on.

INDENTATION IS EVERYTHING IN PYTHON.

The code inside an if block MUST be indented (pushed right) by 4 spaces or 1 Tab.

Python uses indentation to know which lines belong inside the if block.

This is different from most other languages. It catches nearly every beginner.

```
age = 20

if age >= 18:
    print("You are an adult.")          # This runs because 20 >= 18 is True
    print("You can vote in elections.")

print("This always runs.")            # Outside the if block, always
executes
```

Output:

```
You are an adult.
You can vote in elections.
This always runs.
```

Now set age = 15 and run the same code:

Output:

```
This always runs.
```

Notice: when age is 15, the two lines inside the if block are completely skipped. Only the last print runs because it is outside the if block. Python respected the indentation perfectly.

if ... else — two paths

What if you want something to happen when the condition is False as well? Add an else block.

```
marks = 35

if marks >= 40:
    print("Result: PASS")
    print("Congratulations!")
else:
    print("Result: FAIL")
    print("Please appear for the re-exam.")
```

Output:

```
Result: FAIL
Please appear for the re-exam.
```

The if and else blocks are mutually exclusive — only ONE of them will ever run. Python checks the condition, picks one path, runs it, and moves on. The other path is completely ignored.

if ... elif ... else — multiple conditions

When you have more than two possible outcomes, use elif (which stands for 'else if'). Python checks each condition from top to bottom and runs the FIRST one that is True. All the rest are skipped.

```
marks = 73

if marks >= 90:
    grade = "A+"
    remark = "Outstanding!"
elif marks >= 80:
    grade = "A"
    remark = "Excellent!"
elif marks >= 70:
    grade = "B"
    remark = "Good work."
elif marks >= 60:
    grade = "C"
    remark = "Average. Can improve."
elif marks >= 40:
    grade = "D"
    remark = "Just passed. Work harder."
else:
    grade = "F"
    remark = "Failed. Please re-appear."

print(f"Marks: {marks}")
print(f"Grade: {grade}")
print(f"Remark: {remark}")
```

Output:

```
Marks: 73
Grade: B
Remark: Good work.
```

Python checked: is 73 >= 90? No. Is 73 >= 80? No. Is 73 >= 70? YES. It runs the B block and immediately skips all remaining elif and else blocks. Order matters — always put the highest condition first.

Nested if — decisions inside decisions

Sometimes you need to check a second condition only after the first condition is already True. This is called nesting — putting an if inside another if.

```
# Bank loan eligibility check
age = 28
monthly_income = 45000
credit_score = 720

if age >= 21:
    print("Age requirement met.")
    if monthly_income >= 30000:
        print("Income requirement met.")
        if credit_score >= 700:
            print("Loan APPROVED! You qualify for all products.")
        else:
            print("Loan PARTIALLY approved. Credit score too low for premium loans.")
    else:
        print("Loan REJECTED. Income does not meet the minimum requirement.")
else:
    print("Loan REJECTED. Applicant must be at least 21 years old.")
```

Output:

```
Age requirement met.
Income requirement met.
Loan APPROVED! You qualify for all products.
```

Be careful with nesting. More than 3 levels deep becomes very hard to read and maintain. If your code looks like a staircase going far to the right, consider using functions (Section 04) to break it up.

One-line if using the ternary operator

For very simple conditions, Python lets you write an if/else in a single line. This is called a ternary expression. Use it only when the condition is simple — do not sacrifice readability for brevity.

```
# Normal if/else (4 lines)
marks = 55
if marks >= 40:
    result = "PASS"
else:
    result = "FAIL"

# Ternary expression (1 line, same result)
result = "PASS" if marks >= 40 else "FAIL"

print(result)    # PASS
```

Output:

PASS

SECTION 03 Loops — Making Python Repeat Things Automatically

Imagine you need to print the names of 100 students. Writing 100 print statements is not just tedious — it is wrong. Loops let you write the action once and have Python repeat it as many times as needed.

A loop is a block of code that repeats automatically until a certain condition is met. Python has two types: the for loop (repeats a fixed number of times) and the while loop (repeats as long as a condition is True).

The for loop — repeat for each item

The for loop goes through a collection (list, tuple, string, range) one item at a time and runs the indented block for each item. It stops automatically when it reaches the end.

```
students = ["Rahul", "Priya", "Arjun", "Sneha", "Mohammed"]

for student in students:
    print(f"Good morning, {student}!")
```

Output:

```
Good morning, Rahul!
Good morning, Priya!
Good morning, Arjun!
Good morning, Sneha!
Good morning, Mohammed!
```

Read the for loop like plain English: 'For each student in the students list, print good morning to that student.' Python handles all the tracking — you just describe what to do once.

range() — looping a specific number of times

range() generates a sequence of numbers. It is the most common way to repeat something a fixed number of times.

```
# range(5) gives: 0, 1, 2, 3, 4
for i in range(5):
```

```

    print(f"This is repetition number {i}")

print("---")

# range(1, 6) gives: 1, 2, 3, 4, 5
for i in range(1, 6):
    print(f"Question {i}")

print("---")

# range(1, 11, 2) gives: 1, 3, 5, 7, 9 (step of 2)
for i in range(1, 11, 2):
    print(i, end=" ")    # print on same line

```

Output:

```

This is repetition number 0
This is repetition number 1
This is repetition number 2
This is repetition number 3
This is repetition number 4
---
Question 1
Question 2
Question 3
Question 4
Question 5
---
1 3 5 7 9

```

range() form	Generates	Start	End (not included)
range(5)	0, 1, 2, 3, 4	0	5
range(1, 6)	1, 2, 3, 4, 5	1	6
range(0, 10, 2)	0, 2, 4, 6, 8	0	10, step 2
range(10, 0, -1)	10, 9, 8 ... 1	10	0, countdown

Looping with enumerate() — getting the index too

Very often when looping through a list you need both the item AND its position number. `enumerate()` gives you both at the same time.

```

students = ["Rahul", "Priya", "Arjun", "Sneha"]

# Without enumerate — awkward way
# for i in range(len(students)):
#     print(i, students[i])

# With enumerate — clean and Pythonic

```

```
for index, name in enumerate(students):  
    print(f"Student {index + 1}: {name}")
```

Output:

```
Student 1: Rahul  
Student 2: Priya  
Student 3: Arjun  
Student 4: Sneha
```

The while loop — repeat as long as a condition is True

The while loop keeps running its block of code as long as the condition at the top stays True. The moment the condition becomes False, it stops.

```
# Countdown timer  
count = 5  
  
while count > 0:  
    print(f"Launching in {count}...")  
    count = count - 1    # reduce count by 1 each time  
  
print("Liftoff!")
```

Output:

```
Launching in 5...  
Launching in 4...  
Launching in 3...  
Launching in 2...  
Launching in 1...  
Liftoff!
```

The most dangerous bug in programming: the infinite loop.

If you forget to update the condition inside a while loop, it will run forever and freeze your program.

```
count = 5  
while count > 0:  
    print("Running...") # count never changes — this runs FOREVER
```

Always make sure something inside the while loop will eventually make the condition False. If your program freezes, press Ctrl + C in the terminal to force-stop it.

A practical while loop example — password validation:

```

correct_password = "python123"
attempts         = 0
max_attempts     = 3

while attempts < max_attempts:
    entered = input("Enter password: ")
    attempts = attempts + 1

    if entered == correct_password:
        print("Access granted! Welcome.")
        break      # exit the loop immediately
    else:
        remaining = max_attempts - attempts
        if remaining > 0:
            print(f"Wrong password. {remaining} attempts remaining.")
        else:
            print("Account locked. Too many failed attempts.")

```

break, continue, and else in loops

Python gives you three special keywords to control loop behaviour with more precision.

break — exit the loop immediately

```

# Search for a student in the list
students = ["Rahul", "Priya", "Arjun", "Sneha", "Mohammed"]
search_name = "Arjun"

for student in students:
    if student == search_name:
        print(f"Found {search_name} in the list!")
        break      # No need to keep checking, exit the loop
    print(f"Checked: {student}")

```

Output:

```

Checked: Rahul
Checked: Priya
Found Arjun in the list!

```

continue — skip this iteration and go to the next one

```

# Print only students who passed (marks >= 40)
marks_list = [85, 35, 92, 28, 78, 45]

print("Students who passed:")
for marks in marks_list:
    if marks < 40:
        continue    # skip this student, go to next
    print(f"  Marks: {marks} — PASS")

```

Output:

```
Students who passed:
Marks: 85 — PASS
Marks: 92 — PASS
Marks: 78 — PASS
Marks: 45 — PASS
```

else after a loop — runs when loop finishes WITHOUT a break

This is unique to Python and very useful. The else block after a loop runs only if the loop completed normally without hitting a break.

```
students = ["Rahul", "Priya", "Arjun", "Sneha"]
search_name = "Kavya"

for student in students:
    if student == search_name:
        print(f"Found {search_name}!")
        break
else:
    # This runs because the loop finished without a break
    print(f"{search_name} was not found in the list.")
```

Output:

```
Kavya was not found in the list.
```

SECTION 04 Functions — Reusable Blocks of Code

Function

What it is: A named, reusable block of code that performs a specific task. You define it once and call it as many times as you need from anywhere in your program.

Real-life analogy: Think of a washing machine. You do not rebuild it every time you want to wash clothes. You press the button and it does its job. A function is your custom button — you define what it does once and press it (call it) whenever needed.

In Python: `def greet(name): ...` then call it with `greet("Rahul")`

Why do we need functions?

Without functions, if you need the same logic in three places in your program, you copy-paste the code three times. Now imagine finding a bug in that logic — you have to fix it in three places. Functions solve this completely.

Without functions (bad)	With functions (good)
# Repeated code everywhere marks1 = 73 if marks1 >= 90: grade1 = 'A+' elif marks1 >= 80: grade1 = 'A' elif marks1 >= 70: grade1 = 'B' ... marks2 = 55 if marks2 >= 90: grade2 = 'A+' # ... same code again	# Define once, use anywhere def get_grade(marks): if marks >= 90: return 'A+' elif marks >= 80: return 'A' elif marks >= 70: return 'B' ... grade1 = get_grade(73) grade2 = get_grade(55)

Defining and calling a function

There are two separate steps: first you **DEFINE** the function (write what it does), then you **CALL** it (actually run it).

```
# Step 1: DEFINE the function
def greet_student(name):
    print(f"Hello, {name}!")
    print(f>Welcome to the AI/ML program, {name}.")
    print("Let's build something great today.")

# Step 2: CALL the function (as many times as needed)
greet_student("Rahul")
print("---")
greet_student("Priya")
print("---")
greet_student("Arjun")
```

Output:

```
Hello, Rahul!
Welcome to the AI/ML program, Rahul.
Let's build something great today.
---
Hello, Priya!
Welcome to the AI/ML program, Priya.
Let's build something great today.
---
Hello, Arjun!
Welcome to the AI/ML program, Arjun.
Let's build something great today.
```

Anatomy of a function:

```
def      — keyword that tells Python 'I am defining a function'
greet_student — the function name (you choose this)
(name)   — the parameter (the input the function needs)
:        — colon marks the start of the function body
(indent) — everything indented belongs inside the function
```

Parameters and Arguments

A parameter is the variable name inside the function definition (the placeholder). An argument is the actual value you pass when you call the function. Many people use these words interchangeably, but technically they are different.

```
# 'name' and 'marks' are PARAMETERS (placeholders in the definition)
def show_result(name, marks):
    grade = "PASS" if marks >= 40 else "FAIL"
    print(f"{name}: {marks} marks - {grade}")

# 'Rahul' and 73 are ARGUMENTS (actual values passed when calling)
show_result("Rahul", 73)
show_result("Priya", 92)
show_result("Arjun", 35)
```

Output:

```
Rahul: 73 marks - PASS
Priya: 92 marks - PASS
Arjun: 35 marks - FAIL
```

Return values — functions that give back a result

So far our functions have printed things. But most real functions do a calculation and give the result BACK to the caller, using the return keyword. This is what makes functions truly powerful.

```
def calculate_gst(price, gst_rate=0.18):
    gst_amount = price * gst_rate
    total      = price + gst_amount
    return total      # gives the result back to whoever called this

# The returned value is captured in a variable
bill1 = calculate_gst(500)
bill2 = calculate_gst(1200)
bill3 = calculate_gst(800, 0.12)      # 12% GST for this one

print(f"Bill 1: Rs. {bill1}")
print(f"Bill 2: Rs. {bill2}")
print(f"Bill 3: Rs. {bill3}")
```

Output:

```
Bill 1: Rs. 590.0
Bill 2: Rs. 1416.0
Bill 3: Rs. 896.0
```

Default parameter values

You can give a parameter a default value. If the caller does not provide that argument, the default is used automatically. This makes functions flexible.

```
def create_profile(name, role="Student", batch="2024"):
    print(f"Name : {name}")
    print(f"Role : {role}")
    print(f"Batch : {batch}")
    print("----")

create_profile("Rahul")           # Uses both defaults
create_profile("Priya", "Mentor") # Overrides role only
create_profile("Arjun", "Student", "2025") # Overrides both
```

Output:

```
Name : Rahul
Role : Student
Batch : 2024
----
Name : Priya
Role : Mentor
Batch : 2024
----
Name : Arjun
Role : Student
Batch : 2025
----
```

Returning multiple values

Python functions can return more than one value at the same time. The values are packed into a tuple automatically.

```
def analyse_scores(scores):
    highest = max(scores)
    lowest = min(scores)
    average = sum(scores) / len(scores)
    return highest, lowest, average # returns 3 values

class_scores = [85, 92, 78, 95, 88, 70, 63]

# Unpack the returned values into 3 separate variables
top, bottom, avg = analyse_scores(class_scores)

print(f"Highest : {top}")
print(f"Lowest : {bottom}")
print(f"Average : {avg:.2f}")
```

Output:

```
Highest : 95
Lowest : 70
Average : 81.57
```

Docstrings — documenting your functions

A docstring is a short description of what your function does, written as the very first line inside the function using triple quotes. This is professional practice — every function you write should have one.

```
def calculate_bmi(weight_kg, height_m):
    """
    Calculate the Body Mass Index (BMI) of a person.

    Parameters:
        weight_kg (float): Weight of the person in kilograms.
        height_m (float): Height of the person in metres.

    Returns:
        float: The calculated BMI value.
    """
    bmi = weight_kg / (height_m ** 2)
    return round(bmi, 2)

result = calculate_bmi(70, 1.75)
print(f"BMI: {result}")

# You can read the docstring any time using help()
help(calculate_bmi)
```

Output:

```
BMI: 22.86
```

Writing docstrings is a habit that separates professionals from beginners. Your teammates, your future self, and any interviewer looking at your GitHub will appreciate it. Get into this habit from Day 4 itself.

SECTION 05 Modules — Using Other People's Code

Module

What it is: A file containing Python code (functions, variables, classes) that you can import and use in your own program. Python comes with hundreds of built-in modules. You can also install thousands more from the internet.

Real-life analogy: Think of a module like a toolbox. A carpenter does not forge every nail and hammer from scratch. They buy a toolbox with the tools already in it. In Python, a module is a toolbox full of ready-made functions someone else already built for you.

In Python: `import math` then use `math.sqrt(16)`

Importing a module

Python comes with many built-in modules ready to use. You just need to import them at the top of your file. You do not need to install anything for these — they come with Python.

```
import math

print(math.pi)          # 3.141592653589793
print(math.sqrt(144))    # 12.0    (square root)
print(math.ceil(4.3))    # 5      (round UP)
print(math.floor(4.9))   # 4      (round DOWN)
print(math.pow(2, 10))   # 1024.0  (2 to the power of 10)
print(math.fabs(-7.5))   # 7.5    (absolute value, removes minus sign)
```

Output:

```
3.141592653589793
12.0
5
4
1024.0
7.5
```

The random module — generating random values

The random module is used constantly in ML and data science — for shuffling data, splitting datasets, random sampling, and more.

```
import random

# Random integer between 1 and 6 (like rolling a dice)
dice = random.randint(1, 6)
print(f"Dice roll: {dice}")

# Random float between 0.0 and 1.0
probability = random.random()
print(f"Random probability: {probability:.4f}")

# Pick a random item from a list
students = ["Rahul", "Priya", "Arjun", "Sneha", "Mohammed"]
chosen = random.choice(students)
print(f"Today's presenter: {chosen}")

# Shuffle a list randomly (in-place)
random.shuffle(students)
print(f"New order: {students}")

# Pick multiple unique random items
sample = random.sample(students, 3)
print(f"3 random students: {sample}")
```

Output:

```
Dice roll: 4 (yours will differ – it is random!)
Random probability: 0.7823
Today's presenter: Priya
New order: ['Arjun', 'Rahul', 'Sneha', 'Mohammed', 'Priya']
3 random students: ['Mohammed', 'Sneha', 'Arjun']
```

The datetime module — working with dates and times

Any program that handles bookings, logs, deadlines, or timestamps needs datetime. This module is also very commonly used in data science to analyse time-series data.

```
from datetime import datetime, date

# Get the current date and time
now = datetime.now()
print(f"Right now: {now}")
print(f"Year   : {now.year}")
print(f"Month  : {now.month}")
print(f"Day    : {now.day}")
print(f"Hour   : {now.hour}")

# Format date as a readable string
formatted = now.strftime("%d %B %Y, %I:%M %p")
print(f"Formatted: {formatted}")

# Calculate someone's age
birthday = date(2002, 8, 15)
today    = date.today()
age_days = (today - birthday).days
age_years = age_days // 365
print(f"Age: approximately {age_years} years old")
```

Output:

```
Right now: 2024-11-15 09:34:22.123456
Year   : 2024
Month  : 11
Day    : 15
Hour   : 9
Formatted: 15 November 2024, 09:34 AM
Age: approximately 22 years old
```

Two ways to import — and when to use each

Import style	When to use it
<code>import math</code>	When you need many things from the module. Access as <code>math.sqrt()</code> , <code>math.pi</code>

<code>from math import sqrt</code>	When you only need one specific thing. Then use <code>sqrt()</code> directly without the prefix
<code>from math import *</code>	Imports everything. AVOID this — it pollutes your namespace and causes confusion
<code>import numpy as np</code>	Import with an alias. <code>np.array()</code> is shorter than <code>numpy.array()</code> . Standard in data science

```
# Style 1: import the whole module
import math
print(math.sqrt(25))      # 5.0

# Style 2: import specific function only
from math import sqrt, pi
print(sqrt(25))           # 5.0   (no math. prefix needed)
print(pi)                 # 3.141592653589793

# Style 3: import with alias (you will use this constantly in ML)
import random as rnd
print(rnd.randint(1, 100))
```

SECTION 06 Putting It All Together — A Complete Program

This final program combines everything from today: if/elif/else decisions, a for loop, functions, and a module import. Read through it line by line and make sure you understand each part.

```
import random

# — FUNCTION 1: Assign a grade based on marks —————
def get_grade(marks):
    """Return a letter grade and remark for given marks."""
    if marks >= 90:
        return "A+", "Outstanding"
    elif marks >= 80:
        return "A", "Excellent"
    elif marks >= 70:
        return "B", "Good"
    elif marks >= 60:
        return "C", "Average"
    elif marks >= 40:
        return "D", "Passed"
    else:
        return "F", "Failed"

# — FUNCTION 2: Print a formatted result card —————
def print_result_card(name, marks):
    """Print a formatted result card for a student."""
```

```

    grade, remark = get_grade(marks)
    status      = "PASS" if marks >= 40 else "FAIL"
    print(f"  Name   : {name}")
    print(f"  Marks  : {marks} / 100")
    print(f"  Grade   : {grade}")
    print(f"  Status  : {status}")
    print(f"  Remark   : {remark}")
    print("    " + "-" * 30)

# — MAIN PROGRAM —————
students = ["Rahul", "Priya", "Arjun", "Sneha", "Mohammed"]

# Generate random marks for each student
print("==== CLASS RESULT SHEET =====")
print()

all_marks = []
for student in students:
    marks = random.randint(30, 100)    # random marks between 30 and 100
    all_marks.append(marks)
    print_result_card(student, marks)

# Class summary
print("==== CLASS SUMMARY =====")
print(f"  Highest Marks : {max(all_marks)}")
print(f"  Lowest Marks  : {min(all_marks)}")
print(f"  Class Average : {sum(all_marks)/len(all_marks):.1f}")
passed = sum(1 for m in all_marks if m >= 40)
print(f"  Students Passed: {passed} out of {len(students)}")

```

Output:

```

===== CLASS RESULT SHEET =====

  Name   : Rahul
  Marks  : 87 / 100
  Grade   : A
  Status  : PASS
  Remark   : Excellent
  -----
  Name   : Priya
  Marks  : 54 / 100
  Grade   : D
  Status  : PASS
  Remark   : Passed
  -----
  ... (and so on for each student)

===== CLASS SUMMARY =====
  Highest Marks : 94
  Lowest Marks  : 38
  Class Average : 68.4
  Students Passed: 4 out of 5

```

Speak these answers aloud. If you can explain it simply without looking at your notes, you own the concept.

Q — What is the difference between a for loop and a while loop?

A for loop is used when you know exactly how many times you want to repeat something — for example, going through every item in a list or running something 10 times using `range()`. A while loop is used when you do not know in advance how many times you will repeat — you keep going as long as a condition is `True`. For example, keep asking for a password until the user enters the correct one. The risk with while loops is creating an infinite loop if the condition never becomes `False`.

Q — What is a function and why do we use them?

A function is a named, reusable block of code that performs a specific task. We use functions for three main reasons: to avoid repeating the same code in multiple places (DRY — Don't Repeat Yourself), to break a large complex program into smaller, manageable pieces, and to make code easier to test and debug because each function has a single clear responsibility. A function takes inputs through parameters, does something with them, and optionally returns a result.

Q — What is the difference between a parameter and an argument?

A parameter is the variable name in the function definition — it is a placeholder. An argument is the actual value passed when the function is called. For example, in `def greet(name)`, `name` is the parameter. When we call `greet('Rahul')`, the string `'Rahul'` is the argument. Many people use these terms interchangeably in conversation, but technically the distinction is: parameter is in the definition, argument is in the call.

Q — What is a Python module? Can you give examples?

A module is a file containing Python code — functions, variables, and classes — that you can import and use in your own program. Python includes hundreds of built-in modules. For example, `math` provides mathematical functions like `sqrt` and `pi`. `random` provides functions for generating random numbers and shuffling data. `datetime` provides tools for working with dates and times. In data science, we will use external modules like `numpy`, `pandas`, and `scikit-learn` constantly. You import them with the `import` statement at the top of your file.

Term	Definition
Control flow	The ability of a program to make decisions and choose which code to run.
if / elif / else	Keywords for conditional decision-making. Run code only when a condition is <code>True</code> .

Indentation	Spaces at the start of a line that tell Python which block the code belongs to. 4 spaces is standard.
for loop	Repeats a block of code for each item in a collection or a range.
while loop	Repeats a block of code as long as a condition is True.
range()	Generates a sequence of numbers. Used with for loops.
break	Exits a loop immediately, before it has finished all iterations.
continue	Skips the rest of the current iteration and jumps to the next one.
enumerate()	Gives you both the index and value when looping through a list.
Function	A named, reusable block of code defined with def.
Parameter	The variable name in a function definition. A placeholder for the input.
Argument	The actual value passed to a function when calling it.
return	Sends a value back from a function to the code that called it.
Default parameter	A parameter with a pre-set value used when no argument is provided.
Docstring	A description of a function written in triple quotes as its first line.
Module	A file of Python code you can import and use. Example: math, random, datetime.
import	Keyword to load a module into your current program.
DRY principle	Don't Repeat Yourself. If you write the same code twice, it should be a function.
Infinite loop	A while loop that never ends because its condition never becomes False.

SECTION HW Homework — Submit Before Day 5

Create `day4_homework.py` and push it to GitHub before midnight. Every exercise must be inside a properly named function with a docstring.

Exercise 1 — Grade Calculator Function (Beginner)

Write a function called `get_grade(marks)` that takes marks as input and returns the grade letter (A+, A, B, C, D, F) based on the same scale used in today's notes. Then call the function for at least 5 different mark values and print the results.

Remember: the function must use `return`, not just `print`. The caller should be doing the printing.

Exercise 2 — Number Patterns with Loops (Beginner-Intermediate)

Using for loops and range(), print the following three patterns in separate functions: (1) Multiplication table of any number entered by the user (1 to 10). (2) Sum of all numbers from 1 to N, where N is entered by the user. (3) All even numbers between 1 and 50 on a single line.

Exercise 3 — List Processor Function (Intermediate)

Write a function called `analyse_list(numbers)` that takes a list of numbers and returns a dictionary containing: the count, sum, average, maximum, and minimum. Call it with at least two different lists and print the results in a clean format.

Hint: use the built-in functions `len()`, `sum()`, `max()`, `min()` inside your function.

Exercise 4 — Number Guessing Game (Intermediate)

Build a number guessing game using a while loop. The program picks a random number between 1 and 100 (use the random module). The user keeps guessing until they get it right. After each wrong guess, tell them if the actual number is higher or lower. Count how many attempts it took and display it when they win.

Exercise 5 — My Own Function Library (Intermediate)

Create a file called `mystats.py`. In it, write five functions from scratch WITHOUT using any imports: `mean(numbers)`, `find_max(numbers)`, `find_min(numbers)`, `total(numbers)`, and `is_even(number)`. Each must have a docstring. Then in your `day4_homework.py`, import from `mystats` and use all five functions.

Hint for mean: add all numbers together and divide by how many there are.

Hint for importing: `from mystats import mean, find_max, find_min, total, is_even`

This exercise teaches you to write and import your OWN module — exactly what real developers do.

Stretch Challenge — Tier A students

Build a mini student management system using functions, loops, and a list of dictionaries. The system should: (1) have a function to add a student (name, marks), (2) a function to display all students with their grade, (3) a function to find the top scorer, and (4) a function to show how many passed and failed. Run all four functions on a list of at least 6 students.